Studio Nx

P1-02 프론트 공통 JS — 함수 설명 합본 (01~08)

2026-10-12

목차

1. ai-notes/javascript/P1-02 - 프론트 공통 JS/01_api_refreshAccessToken.md
2. ai-notes/javascript/P1-02 - 프론트 공통 JS/02_api_apiFetch.md
3. ai-notes/javascript/P1-02 - 프론트 공통 JS/03_auth_safeNextPath.md
4. ai-notes/javascript/P1-02 - 프론트 공통 JS/04_auth_loginUrl.md
5. ai-notes/javascript/P1-02 - 프론트 공통 JS/05_auth_parseLoginHash.md
6. ai-notes/javascript/P1-02 - 프론트 공통 JS/06_auth_isLoggedIn.md
7. ai-notes/javascript/P1-02 - 프론트 공통 JS/07_auth_requireLogin.md
8. ai-notes/javascript/P1-02 - 프론트 공통 JS/08_auth_logout.md

JS 01 — `refreshAccessToken()`

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
다음	02 `apiFetch()`

1. 하는 일

리프레시 쿠키로 Access Token을 다시 받습니다. 성공하면 `true`, 실패하면 `false`를 돌려줍니다.

2. 언제 쓰나

Access Token은 메모리에만 있어서 **페이지를 옮기면 사라지고**, **30분이 지나면 만료**됩니다. 그때 로그인을 다시 하지 않고 토큰만 새로 받으려고 이 함수를 씁니다.

상황	부르는 곳
페이지를 열고 첫 API 요청을 보낼 때 (토큰이 아직 없음)	`apiFetch()`가 요청 전에 먼저 부릅니다
요청했는데 401이 왔을 때 (토큰 만료)	`apiFetch()`가 재발급한 뒤 같은 요청을 한 번 더 보냅니다
로그인이 필요한 페이지에 들어올 때	`isLoggedIn()` → `requireLogin()` (마이페이지, 내 프로젝트, 회원 정보, 시퀀서)
헤더를 로그인 상태로 바꿀 때	`isLoggedIn()` (모든 페이지의 헤더)

페이지 코드가 이 함수를 직접 부르는 일은 거의 없고, 위의 함수들을 거쳐서 부릅니다.

리프레시 쿠키 (`snx_refresh`)

로그인을 오래 유지하려고 서버가 브라우저에 심는 쿠키이며, 안에 리프레시 토큰이 들어 있습니다. (서버 쪽은 P1-05의 `api/v1/auth.py` 입니다.)

언제	무슨 일
로그인할 때 (`POST /auth/login`)	서버가 쿠키를 심습니다. **자동 로그인을 체크**하면 3일 동안 유지되고(브라우저를 닫아도 남음), **체크하지 않으면** 세션 쿠키가 됩니다(브라우저를 닫으면 사라짐)
재발급할 때 (`POST /auth/refresh`, 이 함수)	브라우저가 쿠키를 자동으로 보내고, 서버가 확인한 뒤 새 Access Token을 줍니다
로그아웃할 때 (`POST /auth/logout`)	서버가 쿠키를 지웁니다
그 밖의 API 요청	쿠키가 **실리지 않습니다**. `path=/api/v1/auth`로 범위를 좁혀 두었기 때문입니다

속성	뜻
`HttpOnly`	JS에서 읽을 수 없습니다. 그래서 스크립트 공격(XSS)으로 훔칠 수 없습니다

속성	뜻
`SameSite=Lax`	다른 사이트가 보낸 POST에는 실리지 않습니다. CSRF에 대비한 설정입니다
`Secure` (배포)	HTTPS에서만 전송합니다
`path=/api/v1/auth`	로그인, 재발급, 로그아웃 요청에만 실립니다

토큰	저장 위치	수명	쓰임
Access Token	메모리 (`api.js`의 변수)	30분	모든 API 요청의 `Authorization: Bearer …`
리프레시 토큰	HttpOnly 쿠키	3일 또는 브라우저를 닫을 때까지	Access Token을 다시 받을 때만 씁니다

쿠키가 없거나 만료되면 `/auth/refresh`가 401로 응답하고, 이 함수는 `false`를 돌려주므로 로그인하지 않은 상태로 처리됩니다.

3. 코드

import (파일 맨 위) `api.js`는 다른 파일을 import하지 않습니다 (공통 모듈의 맨 아래 단계입니다). 이 함수가 쓰는 `refreshing`, `setAccessToken`, `clearAccessToken`, `API_BASE`는 같은 파일 안에 있고, `fetch`는 브라우저의 기본 기능입니다.

함수

```
export async function refreshAccessToken() {
  if (refreshing) return refreshing;
  refreshing = (async () => {
    try {
      const res = await fetch(`${API_BASE}/auth/refresh`, {
        method: 'POST',
        credentials: 'same-origin',
        headers: { Accept: 'application/json' },
      });
      const data = res.ok ? await res.json().catch(() => null) : null;
      if (!data?.access_token) {
        clearAccessToken();
        return false;
      }
      setAccessToken(data.access_token);
      return true;
    } catch {
      clearAccessToken();
      return false;
    } finally {
      refreshing = null;
    }
  })();
  return refreshing;
}
```

문법 풀이

import 문법	뜻
`export` (함수 앞)	다른 파일이 `import { 함수 }`로 가져다 쓸 수 있게 내보냅니다. 없으면 이 파일 안에서만 쓸 수 있습니다

`if (!data?.access_token)`: "토큰을 받지 못했으면"이라는 뜻입니다.

부분	뜻
`data?.access_token`	**옵셔널 체이닝(`?.`)**입니다. `data`가 `null`이나 `undefined`이면 오류 없이 `undefined`가 되고, 아니면 `data.access_token`이 됩니다
`!`	값이 없거나(`undefined`, `null`) 빈 문자열이면 `true`가 됩니다

`data`	`data?.access_token`	`!data?.access_token`	결과
`null` (실패 응답이거나 JSON이 아님)	`undefined`	`true`	실패로 처리합니다
`{}` (토큰 없음)	`undefined`	`true`	실패로 처리합니다
`{ access_token: '' }`	`''`	`true`	실패로 처리합니다
`{ access_token: 'eyJ…' }`	`'eyJ…'`	`false`	성공으로 처리합니다

`?.` 없이 `data.access_token`이라고 쓰면, `data`가 `null`일 때 `TypeError: Cannot read properties of null`로 멈춥니다. 같은 뜻을 길게 쓰면 `if (data === null || data === undefined || !data.access_token)`입니다.

`res.ok ? await res.json().catch(() => null) : null`: 성공 응답(200~299)일 때만 본문을 JSON으로 읽고, 읽다가 실패하면 `null`이 됩니다. 실패 응답이면 읽지 않고 `null`이 됩니다. 그래서 아래의 `if` 하나로 모든 실패를 거를 수 있습니다.

4. 흐름

1. 재발급이 이미 진행 중이면 그 Promise를 돌려줍니다.
2. `POST /api/v1/auth/refresh`를 보냅니다. 쿠키는 브라우저가 자동으로 싣습니다.
3. 응답에 `access_token`이 있으면 저장하고 `true`를 돌려줍니다.
4. 401, 네트워크 오류, 토큰이 없는 응답이면 토큰을 지우고 `false`를 돌려줍니다.
5. 끝나면 `refreshing`을 비웁니다.

5. 왜 이렇게

선택	이유
진행 중인 Promise를 공유 (`refreshing`)	여러 요청이 동시에 401을 받아도 재발급은 한 번만 합니다. 리프레시 토큰을 한 번 쓰고 바꾸는 방식에서도 안전합니다
토큰을 JS가 넣지 않음	리프레시 토큰은 HttpOnly 쿠키라서 JS에서 보이지 않습니다(XSS 대비). `credentials: 'same-origin'`이면 자동으로 전송됩니다
실패해도 예외를 던지지 않음	부르는 쪽(`apiFetch`, `isLoggedIn`)은 성공 여부만 알면 됩니다
`res.json().catch(() => null)`	JSON이 아닌 응답도 예외 없이 실패로 처리합니다
`finally`에서 비움	성공이든 실패든 상관없이, 다음 호출은 새로 요청하게 합니다

6. 시험 (fetch를 가짜로 바꿔서 시험: `/auth/refresh`는 P1-05에서 구현)

경우	결과
200 + `access_token`	`true`를 돌려주고 토큰을 저장합니다. 주소, POST, 쿠키 옵션도 확인했습니다

경우	결과
401	`false`를 돌려주고 토큰을 지웁니다
네트워크 오류	`false`를 돌려주고 토큰을 지우며, 예외는 나지 않습니다
200인데 토큰이 없음	`false`
동시에 3번 호출	fetch는 1번만 하고, 셋 다 `true`입니다
끝난 뒤 다시 호출	새로 요청해서 새 토큰을 받습니다

JS 02 — `apiFetch(path, options)`

항목	내용
파일	`studionx/frontend/static/js/core/api.js` (도우미 `toApiError` 포함)
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	01 `refreshAccessToken()` · 03 `safeNextPath()`

1. 하는 일

서버 API를 부르는 **유일한** 함수입니다. 토큰을 붙이고, 토큰이 만료되면 재발급한 뒤 한 번 다시 보내며, 모든 실패를 `ApiError`로 바꿔 던집니다.

2. 언제 쓰나

모든 페이지와 모듈은 `fetch`를 직접 쓰지 않고, 이 함수(또는 짧은 이름인 `api.get`, `api.post` …)를 씁니다.

예	부르는 곳
`api.post('/auth/login', { email, password, remember }, { auth: false })`	로그인 모달 (P1-07)
`api.get('/users/me')`	회원 정보와 헤더 (P1-10)
`api.post('/users/me/image', formData)`	프로필 이미지 업로드 (P1-10)
`api.get('/projects?page=1')`, `api.post('/projects', { name })`	내 프로젝트 (P1-14)
`api.get('/me/dashboard')`	마이페이지 (P1-17)

옵션	기본값	뜻
`method`	`'GET'`	HTTP 방식입니다
`body`	없음	객체는 JSON으로 바꾸고, `FormData`는 그대로 보냅니다 (파일 업로드)
`auth`	`true`	`false`이면 토큰을 붙이지 않습니다. 회원가입, 로그인, 문의처럼 로그인 전에 쓰는 API에 씁니다

이 함수가 기대고 있는 원리

원리	내용
`Authorization: Bearer <Access Token>`	서버가 "누가 보냈는지" 아는 방법입니다. 로그인이 필요한 API는 이 헤더가 없거나 토큰이 만료되면 **401 `UNAUTHORIZED`**로 응답합니다
서버 오류 형식	모든 실패가 `{ "error": { "code", "message", "details" } }` 형식입니다 (P1-03 `core/errors.py`). 그래서 화면은 `err.code`로 분기하고 `err.message`를 보여 줍니다
`credentials: 'same-origin'`	같은 사이트로 보내는 요청에 쿠키를 싣습니다. 리프레시 쿠키는 `/api/v1/auth` 아래에만 실립니다 (01)

3. 코드

import (파일 맨 위) `api.js`는 다른 파일을 import하지 않습니다 (공통 모듈의 맨 아래 단계입니다). 이 함수가 쓰는 `accessToken`, `refreshAccessToken`, `ApiError`, `toApiError`, `API_BASE`는 같은 파일 안에 있고, `fetch`는 브라우저의 기본 기능입니다.

함수

```
export async function apiFetch(path, options = {}) {
  const { method = 'GET', body, auth = true } = options;

  const headers = { Accept: 'application/json' };
  let payload;
  if (body instanceof FormData) {
    payload = body;
  } else if (body !== undefined && body !== null) {
    headers['Content-Type'] = 'application/json';
    payload = JSON.stringify(body);
  }

  if (auth && !accessToken) await refreshAccessToken();

  const send = async () => {
    const h = auth && accessToken ? { ...headers, Authorization: `Bearer ${accessToken}` } : headers;
    try {
      return await fetch(API_BASE + path, { method, headers: h, body: payload, credentials: 'same-origi
n' });
    } catch {
      throw new ApiError(0, 'NETWORK_ERROR', 'Network error. Please try again.');
    }
  };

  let res = await send();

  if (res.status === 401 && auth) {
    const err = await toApiError(res);
    if (err.code !== 'UNAUTHORIZED' || !(await refreshAccessToken())) throw err;
    res = await send();
  }

  if (!res.ok) throw await toApiError(res);

  if (res.status === 204) return null;
  const text = await res.text();
  return text ? JSON.parse(text) : null;
}

async function toApiError(res) {
  const data = await res.json().catch(() => null);
  const e = data?.error;
  if (e?.code) return new ApiError(res.status, e.code, e.message ?? '', e.details ?? null);
  return new ApiError(res.status, 'INTERNAL_ERROR', 'Something went wrong. Please try again.');
}
```

문법 풀이

import 문법	뜻
`export` (함수 앞)	다른 파일이 `import { 함수 }`로 가져다 쓸 수 있게 내보냅니다. 없으면 이 파일 안에서만 쓸 수 있습니다

식	뜻
`const { method = 'GET', body, auth = true } = options`	**구조 분해와 기본값**입니다. `options.method`가 없으면 `'GET'`, `options.auth`가 없으면 `true`가 됩니다
`body instanceof FormData`	`body`가 `FormData`로 만든 값인지, 즉 파일 업로드인지 확인합니다
`{ ...headers, Authorization: … }`	**펼침**입니다. `headers`를 복사하고 한 칸을 더한 새 객체를 만듭니다. 원래 `headers`는 그대로이므로, 재시도할 때 새 토큰으로 다시 만들 수 있습니다
`auth && accessToken ? A : B`	둘 다 참이면 A(토큰을 붙임), 아니면 B(그대로)입니다
`if (err.code !== 'UNAUTHORIZED' || !(await refreshAccessToken())) throw err`	"만료 오류가 아니거나 재발급이 실패하면 던진다"는 뜻입니다. `||`는 앞이 참이면 뒤를 실행하지 않으므로, 다른 401(예: 로그인 실패)이면 **재발급을 시도하지도 않습니다**
`e.message ?? ''`	**`??`**는 왼쪽이 `null`이나 `undefined`일 때만 오른쪽 값을 씁니다. (`||`와 달리 `''`이나 `0`은 그대로 둡니다)
`text ? JSON.parse(text) : null`	본문이 비었으면 `null`입니다. `res.json()`은 빈 본문에서 오류가 나기 때문에 문자열로 먼저 읽습니다

4. 흐름

1. 옵션을 읽고 본문을 만듭니다 (객체는 JSON 문자열로, `FormData`는 그대로).
2. 로그인이 필요한 요청인데 토큰이 없으면 먼저 재발급합니다.
3. 요청을 보냅니다. 토큰이 있으면 `Authorization` 헤더를 붙입니다. 네트워크가 끊기면 `NETWORK_ERROR`(status 0)가 됩니다.
4. 401 `UNAUTHORIZED`이면 재발급하고, 성공하면 **한 번만** 다시 보냅니다. 재발급이 실패하거나 다른 종류의 401이면 그 오류를 던집니다.
5. 그 밖의 실패(4xx, 5xx)는 서버 형식대로 `ApiError`로 바꿉니다. JSON이 아니면 `INTERNAL_ERROR`가 됩니다.
6. 성공하면, 204이거나 본문이 비었을 때는 `null`을, 아니면 JSON을 돌려줍니다.

5. 왜 이렇게

선택	이유
fetch를 한 곳에서만 씀	토큰, 재발급, 오류 형식 처리를 페이지마다 다시 쓰지 않아도 됩니다
`FormData`이면 `Content-Type`을 붙이지 않음	브라우저가 `multipart/form-data; boundary=…`를 직접 붙여야 서버가 파일을 읽을 수 있습니다. 직접 쓰면 boundary가 빠져서 깨집니다
재시도는 한 번만	재발급한 뒤에도 401이면 무한히 반복하지 않고 바로 실패합니다
401이라도 `UNAUTHORIZED`일 때만 재발급	로그인 실패(`INVALID_CREDENTIALS`) 같은 401은 토큰 문제가 아닙니다
`send()` 안에서 헤더를 매번 만듦	재시도할 때 **새** 토큰이 실립니다
모든 실패를 `ApiError`로 바꿈	화면은 `try { … } catch (err) { if (err.code === 'EMAIL_TAKEN') … }` 한 가지 방식으로 처리할 수 있습니다

선택	이유
실패 본문 변환을 `toApiError`로 분리	401 처리와 일반 실패가 같은 변환을 씁니다

6. 시험 (fetch를 가짜로 바꿔서 시험)

경우	결과
토큰 없이 `/users/me` 요청	refresh → 요청 순서로 보내고, `Bearer T1`, `Accept`, `credentials`가 맞으며, 결과 JSON을 돌려줍니다
객체 body (POST)	`Content-Type: application/json`, 본문 `{"name":"A"}`
`FormData` body	그대로 보내고, `Content-Type`을 붙이지 않습니다
`auth: false`	재발급하지 않고, `Authorization`도 없습니다
401 `UNAUTHORIZED`	refresh 후 같은 요청을 새 토큰으로 보내서 성공합니다 (요청 3번)
401 + 재발급 실패	`ApiError(401, UNAUTHORIZED)`를 던지고 토큰을 지웁니다
재시도도 401	한 번만 재시도한 뒤 던집니다 (요청 3번)
401 `INVALID_CREDENTIALS`	재발급 없이 그대로 던집니다 (요청 1번)
422	`code`, `message`, `details.email`이 그대로 들어 있습니다
500 HTML 응답	`INTERNAL_ERROR`
네트워크 오류	status 0, `NETWORK_ERROR`
204, 빈 본문 200	`null`

JS 03 — `safeNextPath(next, origin)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	02 `apiFetch()` · 04 `loginUrl()`

1. 하는 일

로그인한 뒤 돌아갈 주소(`next`)가 **이 사이트 안의 경로**인지 검사합니다. 안전하면 그대로 돌려주고, 아니면 `null`을 돌려줍니다.

2. 언제 쓰나

로그인이 필요한 페이지(마이페이지, 내 프로젝트, 회원 정보, 시퀀서)에 로그인하지 않고 들어오면 `home.html#login?next=/myPage.html`로 보냅니다. 로그인 모달에서 로그인에 성공하면 `next`로 돌아갑니다.

상황	부르는 곳
주소의 `#login?next=…`를 읽을 때	`parseLoginHash()`(05)를 거쳐 `home.js`의 로그인 모달에서 씁니다
로그인에 성공한 뒤 이동하기 직전	로그인 폼 처리(P1-07)에서 한 번 더 확인합니다

막는 공격: 열린 리다이렉트 (open redirect)

`next`는 주소창에 그대로 들어 있어서 **누구나 바꿀 수 있습니다**. 검사가 없으면 다음과 같은 일이 생깁니다.

1. 공격자가 `https://studionx…/home.html#login?next=https://evil.com/fake-login` 링크를 보냅니다.
2. 사용자는 진짜 Studio Nx 주소라고 믿고 로그인합니다.
3. 로그인 직후 가짜 사이트로 이동하고, "비밀번호를 다시 입력하세요" 같은 피싱 화면을 보게 됩니다.

그래서 **같은 사이트의 경로만** 허용합니다.

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. **이 함수는 import한 것을 쓰지 않습니다.** `URL`과 `location`은 브라우저의 기본 기능입니다.

함수

```
export function safeNextPath(next, origin = location.origin) {
  if (typeof next !== 'string' || !next.startsWith('/')) return null;
```

```
  if (next.startsWith('//') || next.includes('\\')) return null;
  try {
    return new URL(next, origin).origin === origin ? next : null;
  } catch {
    return null;
  }
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다
`export` (함수 앞)	다른 파일이 `import { 함수 }`로 가져다 쓸 수 있게 내보냅니다. 없으면 이 파일 안에서만 쓸 수 있습니다

식	뜻
`origin = location.origin`	기본값 매개변수입니다. 보통은 지금 사이트(`http://localhost:8000`)이고, 시험할 때만 다른 값을 넣습니다
`typeof next !== 'string'`	문자열이 아니면(`null`, `undefined`, 숫자) 거부합니다
`'\'`	자바스크립트 문자열 안에서 역슬래시 한 글자 `\`를 뜻합니다
`new URL(next, origin)`	`next`를 `origin` 기준으로 해석한 완전한 주소입니다. `new URL('/a', 'http://x')`는 `http://x/a`가 됩니다
`.origin === origin`	해석한 주소의 `프로토콜://호스트:포트`가 이 사이트와 같은지 확인합니다
`try … catch`	이상한 문자열 때문에 `new URL`이 오류를 내면 `null`을 돌려줍니다

4. 흐름

1. 문자열이 아니거나 `/`로 시작하지 않으면 거부합니다. `https://…`, `javascript:…`, `myPage.html`이 여기서 걸립니다.
2. `//`로 시작하거나 `\`가 들어 있으면 거부합니다. 브라우저는 이런 값을 **다른 사이트의 주소**로 읽습니다.
3. 마지막으로 브라우저와 같은 방식(`new URL`)으로 해석해서 이 사이트인지 확인합니다.

5. 왜 이렇게

선택	이유
`/`로 시작하는 경로만 허용	가장 단순하고 확실한 기준입니다. 전체 주소(`http://…`)는 이 사이트의 주소여도 거부합니다. 화면은 경로만 넘기기 때문입니다
`//evil.com` 거부	"프로토콜 상대 주소"라서 브라우저는 `https://evil.com`으로 이동합니다
`\`가 들어가면 거부	브라우저는 `/\evil.com`의 `\`를 `/`로 바꿔서 `//evil.com`으로 읽습니다
`new URL`로 한 번 더 확인	위의 규칙이 놓친 특이한 형태도 실제 해석 결과로 막습니다
오류 대신 `null`을 돌려줌	부르는 쪽은 `null`이면 기본 페이지로 보내면 됩니다

6. 시험

입력	결과
`/myPage.html`, `/studioNx.html?projectId=3`, `/myProject.html#new`	그대로 돌려줍니다
`//evil.com`, `/\evil.com`	`null`
`https://evil.com/x`, `http://localhost:8000/myPage.html`	`null` (전체 주소는 모두 거부합니다)
`javascript:alert(1)`, `myPage.html`, `''`	`null`
`null`, `undefined`, `42`	`null`
`/%2F%2Fevil.com`	그대로 돌려줍니다. 해석하면 `http://localhost:8000/%2F%2Fevil.com`(이 사이트 안의 경로)이라서 안전합니다

JS 04 — `loginUrl(next)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	03 `safeNextPath()` · 05 `parseLoginHash()`

1. 하는 일

로그인 모달로 가는 주소를 만듭니다. 돌아올 경로가 있으면 `next`로 붙입니다.

`loginUrl('/myPage.html')` → `home.html#login?next=%2FmyPage.html`

2. 언제 쓰나

로그인 화면은 따로 없고 **`home.html`의 모달**입니다 (2026-10-02 결정). 그래서 "로그인하러 가기"는 항상 이 주소로 갑니다.

상황	부르는 곳
로그인이 필요한 페이지에 로그인하지 않고 들어옴	`requireLogin()`(07)이 `loginUrl(location.pathname + location.search)`로 부릅니다
회원가입을 마친 뒤 로그인하러 감	`join.js`(P1-07)가 `loginUrl()`로 부릅니다

주소의 `#` 뒷부분 (해시)

부분	쓰임
`#login`	`home.js`가 이것을 보고 로그인 모달을 자동으로 엽니다
`?next=…`	로그인에 성공한 뒤 돌아갈 경로입니다. `#` 뒤에 있으므로 **서버로 보내지지 않고** 브라우저 안에서만 쓰입니다

보안: 주소에 `next`가 보여도 괜찮은가?

한 줄 요약: 주소에 붙는 것은 "어디로 가려 했는지"를 적은 쪽지뿐이고, "열쇠"(토큰, 비밀번호)는 절대 주소에 붙지 않습니다.

비유: 건물 출입

건물	Studio Nx
3층 사무실에 가려는데 출입증이 없습니다	로그인하지 않고 `/myPage.html`에 들어옵니다
안내 데스크로 보내면서 "3층에 가려던 분"이라는 **쪽지**를 줍니다	`home.html#login?next=/myPage.html`로 보냅니다
안내 데스크에서 신분을 확인하고 **출입증**을 받습니다	로그인해서 토큰을 받습니다
쪽지를 보고 3층으로 안내합니다	`next`를 읽고 `/myPage.html`로 이동합니다

쪽지(`next`)는 누가 봐도 괜찮습니다. 가려던 층만 적혀 있기 때문입니다. 출입증(토큰)은 주머니(메모리, HttpOnly 쿠키)에만 넣고, 쪽지에는 절대 적지 않습니다.

순서와 주소창

단계	주소창
① 로그인하지 않고 마이페이지에 들어옴	`/myPage.html`
② 로그인하러 보냄: **이때** `next`가 붙습니다 (로그인 전)	`home.html#login?next=%2FmyPage.html`
③ 모달에 이메일과 비밀번호를 입력해서 성공	(그대로)
④ `next`를 검사하고 원래 페이지로 이동	`/myPage.html` (`next`는 사라집니다)

걱정과 답

걱정	답
주소에 비밀 정보가 남나요?	아닙니다. 남는 것은 경로뿐이고, 토큰은 메모리와 쿠키에만 있습니다
비밀번호가 주소에 붙나요?	아닙니다. 로그인은 `POST` 본문으로 보냅니다 (주소가 아니라 봉투 안에 넣는 셈입니다)
서버나 다른 사이트로 새어 나가나요?	아닙니다. `#` 뒷부분은 브라우저 안에만 있고 서버로 가지 않습니다
누가 `next`를 가짜 사이트로 바꾸면요?	`safeNextPath`가 이 사이트의 경로만 통과시킵니다 (03)
`next`에 스크립트를 넣으면요?	`javascript:` 같은 값은 거부합니다. 그리고 `next`는 **이동에만** 쓰고 화면에 HTML로 넣지 않습니다
`projectId=3`이 보이면요?	번호는 비밀이 아닙니다. 남의 프로젝트는 서버가 권한으로 막습니다 (P1-13)

구현할 때 지킬 것 (P1-07 로그인 폼)

1. `next`는 `location.href = next`처럼 **이동에만** 씁니다. `innerHTML`에 넣지 않습니다.
2. 로그인 폼은 JS로 기본 제출을 막고 `apiFetch`로 보냅니다. 폼에 `method="post"`가 있으므로 JS가 실패해도 비밀번호가 주소에 붙지 않습니다.

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. **이 함수는 import한 것을 쓰지 않습니다.** `safeNextPath`와 `LOGIN_PAGE`는 같은 파일 안에 있습니다.

함수

```
export function loginUrl(next) {
  const path = safeNextPath(next);
  return path ? `${LOGIN_PAGE}#login?next=${encodeURIComponent(path)}` : `${LOGIN_PAGE}#login`;
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다

식	뜻
`encodeURIComponent(path)`	주소에서 특별한 뜻을 가진 글자를 `%XX`로 바꿉니다. `/` → `%2F`, `?` → `%3F`, `=` → `%3D`, `&` → `%26`
`` `${LOGIN_PAGE}#login?next=${…}` ``	**템플릿 문자열**입니다. `${ }` 안의 값을 끼워 넣은 문자열을 만듭니다
`path ? A : B`	안전한 경로가 있으면 A(`next`를 붙임), 없으면 B(`#login`만)입니다

4. 흐름

1. `safeNextPath`로 경로를 검사합니다 (03).
2. 안전하면 인코딩해서 `#login?next=` 뒤에 붙입니다.
3. 경로가 없거나 안전하지 않으면 `home.html#login`만 돌려줍니다.

5. 왜 이렇게

선택	이유
`encodeURIComponent`	경로 안의 `?`, `&`, `=`가 바깥쪽 `?next=`의 구분자로 잘못 읽히지 않게 합니다. 예를 들어 `/myProject.html?q=a&page=2`의 `&page=2`가 `next` 밖으로 새지 않습니다
주소를 만들 때도 `safeNextPath`로 검사	지금은 이 프로젝트의 코드만 부르지만, 잘못된 값이 들어와도 위험한 주소를 만들지 않습니다 (읽을 때도 한 번 더 검사합니다)
`LOGIN_PAGE` 상수	로그인 페이지가 바뀌어도 한 곳만 고치면 됩니다

6. 시험

입력	결과
`/myPage.html`	`home.html#login?next=%2FmyPage.html`
`/studioNx.html?projectId=3`	`…next=%2FstudioNx.html%3FprojectId%3D3`
`/myProject.html?q=a&page=2`	`…next=%2FmyProject.html%3Fq%3Da%26page%3D2`이고, 다시 읽으면 원래 경로 그대로입니다
없음, `''`	`home.html#login`
`https://evil.com`, `//evil.com`	`home.html#login` (`next`를 붙이지 않습니다)

JS 05 — `parseLoginHash(hash)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	04 `loginUrl()` · 06 `isLoggedIn()`

1. 하는 일

주소의 `#login?next=…`를 읽어서, **모달을 열지 여부(`open`)**와 **로그인한 뒤 갈 곳(`next`)**을 돌려줍니다.

`parseLoginHash('#login?next=%2FmyPage.html')` → `{ open: true, next: '/myPage.html' }`

2. 언제 쓰나

04 `loginUrl()`은 로그인 주소를 만들고, 이 함수는 `home.html`에서 그 주소를 읽습니다. 두 함수는 같은 주소 형식(`#login?next=…`)을 쓰므로 바꿀 때는 함께 바꿔야 합니다.

상황	부르는 곳
`home.html`을 열 때	`home.js`의 `initLoginModalFromHash()`(P1-07)가 부릅니다. `open`이면 로그인 모달을 열고 `next`를 기억합니다
로그인에 성공한 뒤	로그인 폼 처리(P1-07)에서 `next`가 있으면 그곳으로, 없으면 기본 페이지로 보냅니다

주소	모달	로그인 후
`home.html`	열리지 않습니다	—
`home.html#login` (헤더의 Sign In, 가입 완료)	열립니다	기본 페이지
`home.html#login?next=%2FmyPage.html` (`requireLogin`)	열립니다	`/myPage.html`

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. **이 함수는 import한 것을 쓰지 않습니다.** `safeNextPath`는 같은 파일에 있고, `URLSearchParams`와 `location`은 브라우저의 기본 기능입니다.

함수

```
export function parseLoginHash(hash = location.hash) {
  if (typeof hash !== 'string' || !hash.startsWith('#login')) return { open: false, next: null };
  const rest = hash.slice('#login'.length);
```

```
  if (rest && !rest.startsWith('?')) return { open: false, next: null };
  const next = new URLSearchParams(rest.slice(1)).get('next');
  return { open: true, next: safeNextPath(next) };
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다

식	뜻
`hash = location.hash`	기본값입니다. 지금 주소의 `#`부터 끝까지입니다 (예: `'#login?next=%2FmyPage.html'`)
`hash.slice('#login'.length)`	앞의 `#login`(6글자)을 잘라낸 나머지입니다. `''` 또는 `'?next=…'`가 됩니다
`rest && !rest.startsWith('?')`	나머지가 있는데 `?`로 시작하지 않으면, `#loginHelp` 같은 **다른** 해시라는 뜻입니다
`rest.slice(1)`	맨 앞의 `?`를 뗀 `next=%2FmyPage.html`입니다
`new URLSearchParams(…).get('next')`	`이름=값&…`을 읽어서 `next` 값을 꺼냅니다. **인코딩을 자동으로 풀어 줍니다** (`%2F` → `/`, `%3F` → `?`). 없으면 `null`입니다

hash	rest	결과
`'#login'`	`''`	`{ open: true, next: null }`
`'#login?next=%2FmyPage.html'`	`'?next=%2FmyPage.html'`	`{ open: true, next: '/myPage.html' }`
`'#loginHelp'`	`'Help'`	`{ open: false, next: null }`
`'#features'`	—	`{ open: false, next: null }`

4. 흐름

1. `#login`으로 시작하지 않으면 모달을 열지 않습니다.
2. `#login` 뒷부분이 비었거나 `?`로 시작하는지 확인합니다 (`#loginHelp`를 거릅니다).
3. `?` 뒤에서 `next`를 꺼냅니다 (인코딩이 풀립니다).
4. `safeNextPath`로 검사합니다. 위험하면 `next: null`이 되지만 **모달은 엽니다**.

5. 왜 이렇게

선택	이유
해시(`#`)에 둠	서버로 가지 않고, 페이지를 다시 불러오지도 않습니다 (04의 보안 설명)
`URLSearchParams`	`&`, `=`, `%XX`를 직접 자르면 실수하기 쉽습니다. 그래서 브라우저 표준 방식으로 읽습니다
읽을 때도 `safeNextPath`로 검사	주소는 누구나 바꿀 수 있습니다. **만들 때 검사했더라도 읽을 때 다시** 검사합니다
위험한 `next`여도 `open: true`	로그인 자체는 정상이므로, 갈 곳만 버리고 기본 페이지로 보냅니다
결과를 객체 하나로 돌려줌	부르는 쪽이 `const { open, next } = parseLoginHash()` 한 줄로 받을 수 있습니다

6. 시험

입력	결과
`#login`	열리고, `next`는 없습니다
`#login?next=%2FmyPage.html`	열리고, `/myPage.html`입니다
`#login?next=%2FstudioNx.html%3FprojectId%3D3`	열리고, `/studioNx.html?projectId=3`입니다
`#login?next=%2FmyProject.html%3Fq%3Da%26page%3D2`	열리고, `/myProject.html?q=a&page=2`입니다
`#login?next=https%3A%2F%2Fevil.com`, `…%2F%2Fevil.com`	열리지만 `next: null`입니다 (풀면 `https://…`, `//evil.com`이라서 거부합니다)
`#login?foo=1`	열리고, `next: null`입니다
`#loginHelp`, `#features`, `''`	열리지 않습니다
인자 없음 (`location.hash` 사용)	`location.hash`를 읽습니다
`loginUrl()`로 만든 주소를 이 함수로 읽기	원래 경로가 그대로 돌아옵니다 (왕복)

JS 06 — `isLoggedIn()`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	05 `parseLoginHash()` · 07 `requireLogin()`

1. 하는 일

지금 로그인 상태인지를 `true` 또는 `false`로 알려 줍니다. 토큰이 없으면 리프레시 쿠키로 다시 받아 본 뒤 판단합니다.

2. 언제 쓰나

상황	부르는 곳
로그인이 필요한 페이지(마이페이지, 내 프로젝트, 회원 정보, 시퀀서)에 들어올 때	07 `requireLogin()`이 부르고, `false`이면 로그인 화면으로 보냅니다
페이지를 열 때 맨 위 메뉴 줄(헤더) 오른쪽에 어떤 버튼을 보여 줄지 정할 때	`auth-ui.js`의 `initAuthState()`(P1-07)가 부릅니다. 로그인 상태면 **My Page, My Projects** 버튼을, 아니면 **Sign In, Join Us** 버튼을 보여 줍니다
`home.html`을 열 때	이미 로그인 상태라면 `#login` 모달을 열지 않습니다 (P1-07)

헤더 버튼이 바뀌는 예

헤더는 홈, 가입, 마이페이지 등 모든 페이지 맨 위에 똑같이 들어가는 메뉴 줄입니다 (`static/partials/header.html`). 오른쪽 버튼 네 개가 모두 HTML에 들어 있고, 이 함수의 결과에 따라 두 개만 보이게 합니다.

로그인 상태	보이는 버튼	숨기는 버튼
로그인하지 않음 (`false`)	Sign In, Join Us	My Page, My Projects
로그인함 (`true`)	My Page, My Projects	Sign In, Join Us

버튼에는 `data-auth-show="out"`(로그인하지 않았을 때 보임) 또는 `data-auth-show="in"`(로그인했을 때 보임) 표시가 붙어 있습니다.

로그인 상태를 판단하는 기준

서버에는 "로그인 중"이라는 기록이 없습니다. 브라우저가 가진 토큰으로 판단합니다.

브라우저 상태	판단
메모리에 Access Token이 있습니다	로그인 상태입니다
Access Token은 없지만 리프레시 쿠키가 유효합니다 (페이지를 옮긴 직후, 자동 로그인)	재발급에 성공하면 로그인 상태입니다
둘 다 없거나 리프레시 쿠키가 만료되었습니다	로그인하지 않은 상태입니다

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. 이 함수는 이 중에서 **`getAccessToken`과 `refreshAccessToken`**을 씁니다 (01).

함수

```
export async function isLoggedIn() {
  if (getAccessToken()) return true;
  return refreshAccessToken();
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다

식	뜻
`async function`	결과를 Promise로 돌려주는 함수입니다. 부르는 쪽은 `await isLoggedIn()`으로 `true` 또는 `false`를 받습니다
`if (getAccessToken())`	토큰 문자열이 있으면 참이고, `null`이면 거짓입니다
`return refreshAccessToken();`	재발급 결과(`true` 또는 `false`)를 그대로 돌려줍니다. `async` 함수 안에서 Promise를 돌려주면 그 결과가 이 함수의 결과가 됩니다

4. 흐름

1. 메모리에 Access Token이 있으면 서버에 묻지 않고 `true`를 돌려줍니다.
2. 없으면 `refreshAccessToken()`을 부릅니다 (01).
3. 재발급에 성공하면 `true`, 실패하면 `false`를 돌려줍니다.

5. 왜 이렇게

선택	이유
토큰이 있으면 바로 `true`	같은 페이지 안에서 여러 번 불려도 서버 요청이 생기지 않습니다
토큰의 만료 시간은 확인하지 않음	만료된 토큰으로 API를 부르면 `apiFetch()`가 401을 받고 재발급한 뒤 다시 보냅니다. 같은 처리를 두 곳에 두지 않기 위해서입니다
재발급은 `refreshAccessToken()`에 맡김	헤더 버튼을 정하는 코드와 `requireLogin()`이 페이지를 열 때 동시에 불러도 재발급 요청은 한 번만 나갑니다 (01의 진행 중인 Promise 공유)
실패해도 예외를 던지지 않음	네트워크 오류도 "로그인하지 않음"으로 봅니다. 부르는 쪽은 `true`와 `false`만 확인하면 됩니다

6. 시험 (fetch를 가짜로 바꿔서 시험)

경우	결과
메모리에 토큰이 있음	`true`이고, 서버 요청은 0번입니다
토큰은 없고 리프레시 쿠키가 유효함	`true`이고, 재발급을 1번 하며, 새 토큰을 저장합니다
토큰도 리프레시 쿠키도 없음 (401)	`false`이고, 토큰이 없습니다
네트워크 오류	`false`이고, 예외는 나지 않습니다
동시에 두 번 호출 (헤더 버튼 정하기 + `requireLogin`)	둘 다 `true`이고, 재발급은 1번입니다

JS 07 — `requireLogin()`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	06 `isLoggedIn()` · 08 `logout()`

1. 하는 일

로그인이 필요한 페이지의 맨 처음에 부르는 함수입니다. 로그인 상태면 `true`를 돌려주고, 아니면 로그인 화면으로 보낸 뒤 `false`를 돌려줍니다.

2. 언제 쓰나

로그인한 사람만 볼 수 있는 페이지 네 곳이 페이지 코드의 맨 처음에 이 함수를 부릅니다.

페이지	부르는 곳
마이페이지 `myPage.html`	`pages/mypage.js` (P1-17)
내 프로젝트 `myProject.html`	`pages/myproject.js` (P1-14)
회원 정보 `userInfo.html`	`pages/userinfo.js` (P1-10)
시퀀서 `studioNx.html`	`pages/sequencer.js`의 `init()`

```
const ok = await requireLogin();
if (!ok) return;          // 로그인 화면으로 이동 중 — 아래 코드(대시보드 불러오기 등)를 실행하지 않음
```

실제로 일어나는 일 (예)

로그아웃한 상태에서 주소창에 `localhost:8000/myProject.html#new`를 직접 입력하면 다음과 같이 진행됩니다.

순서	화면과 주소창
① 내 프로젝트 페이지가 열리기 시작합니다	`/myProject.html#new`
② `requireLogin()`이 로그인하지 않은 상태임을 확인합니다	—
③ 홈으로 이동하고 로그인 모달이 열립니다	`home.html#login?next=%2FmyProject.html%23new`
④ 로그인에 성공합니다 (P1-07)	—
⑤ 원래 가려던 곳으로 돌아오고, 새 프로젝트 모달까지 열립니다	`/myProject.html#new`

로그인한 상태라면 ②에서 `true`가 나오고 페이지가 그대로 이어집니다.

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. **이 함수는 import한 것을 직접 쓰지 않습니다.** `isLoggedIn`과 `loginUrl`은 같은 파일 안에 있고, 그 함수들이 `api.js`를 씁니다.

함수

```
export async function requireLogin() {
  if (await isLoggedIn()) return true;
  location.replace(loginUrl(location.pathname + location.search + location.hash));
  return false;
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다

식	뜻
`await isLoggedIn()`	로그인 상태 확인이 끝날 때까지 기다린 뒤 `true` 또는 `false`를 받습니다 (06)
`location.pathname`	주소의 경로입니다. 예: `/myProject.html`
`location.search`	`?`부터 시작하는 부분입니다. 예: `?projectId=3` (없으면 `''`)
`location.hash`	`#`부터 시작하는 부분입니다. 예: `#new` (없으면 `''`)
`location.replace(url)`	`url`로 이동하면서 **지금 페이지를 방문 기록에서 지웁니다**

4. 흐름

1. `isLoggedIn()`이 `true`이면 바로 `true`를 돌려주고, 페이지는 계속 진행됩니다.
2. 아니면 지금 주소(경로, 쿼리, 해시)로 로그인 주소를 만듭니다 (04 `loginUrl()`).
3. 그 주소로 `location.replace`하고 `false`를 돌려줍니다.

5. 왜 이렇게

선택	이유
`location.href` 대신 `location.replace`	`href`로 이동하면 로그인 화면에서 뒤로 가기를 눌렀을 때 마이페이지로 돌아갔다가 **다시 로그인 화면으로 튕기는 일이 반복됩니다**. `replace`는 마이페이지를 방문 기록에서 지우므로, 뒤로 가기를 누르면 그 전 페이지로 갑니다
쿼리와 해시까지 넣음	`?projectId=3`(어떤 프로젝트인지)이나 `#new`(새 프로젝트 모달)가 빠지면 로그인한 뒤 엉뚱한 화면이 나옵니다. 원래 템플릿은 경로와 쿼리만 넣었는데, 해시도 더했습니다
`false`를 돌려줌	이동은 바로 끝나지 않습니다. 그사이 페이지 코드가 API를 부르지 않도록 `if (!ok) return;`으로 멈추게 합니다
판단은 `isLoggedIn()`에 맡김	토큰 처리와 재발급 처리를 다시 쓰지 않아도 됩니다

6. 시험 (fetch와 location을 가짜로 바꿔서 시험)

경우	결과
토큰 있음	`true` 이고, 이동하지 않습니다
토큰은 없고 리프레시 쿠키가 유효함	`true` 이고, 이동하지 않습니다
로그인하지 않은 상태로 `/myPage.html`	`false` 이고, `home.html#login?next=%2FmyPage.html` 로 이동합니다
`/studioNx.html?projectId=3`	`…next=%2FstudioNx.html%3FprojectId%3D3`
`/myProject.html#new`	`…next=%2FmyProject.html%23new`
위의 주소를 `parseLoginHash()` 로 다시 읽기	`/myProject.html#new` 가 그대로 나옵니다

JS 08 — `logout()`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-104)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	07 `requireLogin()` (P1-02의 마지막 함수) · 09 `setFieldState()` (P1-07)

1. 하는 일

로그아웃합니다. 서버에 리프레시 쿠키를 지워 달라고 요청하고, 메모리의 Access Token을 지운 뒤 홈으로 이동합니다.

2. 언제 쓰나

마이페이지, 내 프로젝트, 회원 정보 화면의 **왼쪽 메뉴 맨 아래에 있는 `Log out` 버튼**(`#logout-button`)을 누를 때 부릅니다.

페이지	부르는 곳
마이페이지 `myPage.html`	`pages/mypage.js` (P1-17)
내 프로젝트 `myProject.html`	`pages/myproject.js` (P1-14)
회원 정보 `userInfo.html`	`pages/userinfo.js` (P1-10)

누르면 일어나는 일 (예)

순서	화면과 브라우저
① 마이페이지에서 `Log out`을 누릅니다	—
② 서버가 리프레시 쿠키(`snx_refresh`)를 지웁니다	다음에 페이지를 열어도 재발급이 되지 않습니다
③ 메모리의 Access Token을 지웁니다	지금 페이지에서도 API를 부를 수 없습니다
④ 홈으로 이동합니다	`home.html`로 가고, 헤더 오른쪽이 **Sign In, Join Us** 버튼으로 바뀝니다
⑤ 뒤로 가기를 누릅니다	마이페이지가 아니라 그 전에 보던 페이지로 갑니다

토큰이 두 개이기 때문에 로그아웃할 때도 두 곳에서 지웁니다.

토큰	지우는 곳
리프레시 토큰 (HttpOnly 쿠키)	서버가 지웁니다. JS는 이 쿠키를 읽거나 지울 수 없습니다
Access Token (메모리)	이 함수가 `clearAccessToken()`으로 지웁니다

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. 이 함수는 이 중에서 **`api`와 `clearAccessToken`**을 씁니다. `LOGIN_PAGE`는 같은 파일 안에 있습니다.

함수

```
export async function logout() {
  try {
    await api.post('/auth/logout', undefined, { auth: false });
  } catch {
    // 이미 만료 · 네트워크 오류여도 화면에서는 로그아웃을 계속함
  }
  clearAccessToken();
  location.replace(LOGIN_PAGE);
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다

식	뜻
`api.post('/auth/logout', undefined, { auth: false })`	본문 없이(`undefined`) POST를 보냅니다. `auth: false`이므로 토큰을 붙이지 않고, 토큰이 없어도 재발급을 먼저 시도하지 않습니다 (02)
`try { … } catch { }`	요청이 실패해도 오류로 멈추지 않고 다음 줄로 넘어갑니다. `catch` 뒤의 `(err)`는 쓰지 않을 때 생략할 수 있습니다
`location.replace(LOGIN_PAGE)`	`home.html`로 이동하면서 지금 페이지를 방문 기록에서 지웁니다

4. 흐름

1. `POST /api/v1/auth/logout`을 보내고, 서버가 리프레시 쿠키를 지웁니다.
2. 요청이 실패해도 그대로 진행합니다.
3. 메모리의 Access Token을 지웁니다.
4. `home.html`로 이동합니다 (방문 기록에서 지금 페이지를 지웁니다).

5. 왜 이렇게

선택	이유
`auth: false`	서버는 토큰을 확인하지 않고 쿠키만 지웁니다 (P1-05 템플릿). 토큰이 없을 때 재발급을 먼저 하면 **로그아웃하려다가 오히려 새 토큰을 받는** 일이 생깁니다
요청이 실패해도 계속 진행	사용자는 로그아웃 버튼을 눌렀습니다. 네트워크가 끊겨도 화면에서는 로그아웃된 상태가 되어야 합니다
`location.href` 대신 `location.replace`	`href`를 쓰면 뒤로 가기로 마이페이지가 다시 보일 수 있습니다(브라우저가 저장해 둔 화면). `replace`는 그 기록을 지웁니다. 템플릿에는 `href`로 되어 있었는데 바꿨습니다

선택	이유
메모리 토큰은 마지막에 지움	요청을 보내는 동안에는 필요하지 않지만, 실패 여부와 상관없이 반드시 지우도록 `try` 바깥에 두었습니다

6. 시험 (fetch와 location을 가짜로 바꿔서 시험)

경우	결과
정상 (204)	`POST /api/v1/auth/logout`을 1번 보내고, `credentials`는 있고 `Authorization`은 없으며, 토큰을 지우고 `home.html`로 replace합니다
토큰이 이미 없음	재발급을 시도하지 않고 로그아웃 요청만 1번 보냅니다
서버 오류 500	예외 없이 토큰을 지우고 이동합니다
네트워크 오류	예외 없이 토큰을 지우고 이동합니다